

Supplementary Material for “Super-Precise Latency: How CPU Threading Affects High-Precision Latency, and an Industry-Wide Audit”

Gustavo Pedro Ricou and Romaric Duvignau

This document is the supplementary material for the paper named above, which points to the sections here that hold its evidence. Timestamps and spans are those of Equation 1 and Fig. 1 of the paper, and both processes read one clock unless a result says otherwise. Table S1 defines the terms used here that the paper does not. The complete record of how the results were obtained, with every result that did not survive, is a separate document in the archived artifact, the postmortem. Pointers such as “postmortem, S37” refer to its sections.

S1. THE SETUP IN FULL

S1.1. Testbeds, clocks and workload

Two testbeds carry the evidence (Table S2). On the cloud testbed the producer and the consumer run as separate processes on one host with 8 online CPUs, and read one wall clock, `CLOCK_REALTIME`, through `time.time_ns()`. No scheduler tunable is written anywhere in the archived campaign, which is strong evidence, short of proof, that the kernel’s defaults applied, among them the derived base slice of 3 ms (Section IV-C). The clocksource was not recorded, but a measured read cost narrows down which one it was (S3). Across hosts, `chrony` reports an inter-host offset of 0.067 ms but bounds its own error at 4–7 ms per host over the 4 hosts captured, far more than the offset, so we claim no cross-host bound.

The producer replays StatsBomb’s open football event data [S1]. On the cloud testbed it replays eleven of its 3,315 matches, in order, at a chosen publish rate. The registered campaign replays a steady 50 messages a second, except in its block M0, which replays a match’s bursts.

S1.2. Every setting that can move a latency number

Table S3 states every client setting that can move a latency number, and why it holds its value.

S1.3. Where each timestamp is taken, and which spans fall below zero

The publish timestamp is read on the calling thread immediately before the client is invoked. The two clients read the acknowledgment timestamp in different places, but in both it is read only after a thread that was not running has been woken and then given a core. Under Redis it is read on the calling thread the moment the synchronous `xadd` returns, with no second

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.

R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.

TABLE S1
TERMS USED HERE THAT THE PAPER DOES NOT DEFINE.

Term	Meaning
condition	One point of the experimental design: a broker, a publish rate, a message size, a background load and a consumer pattern, which is how often the consumer confirms what it has read.
cell	A condition together with a workload; where the workload is fixed, the two coincide. A run executes a cell once, and a replicate is a further run of it.
configuration	One setting of a manipulation that compares settings differing in one variable.
core count k	The number of cores the background load occupies.
load geometry	How a utilization is reached: the load concentrated on few cores or spread over many.
grid	The instants, τ apart, at which a timestamp changes value.
q -lattice	The retentions $0, 1/q, \dots, 1$ a paced producer can reach when its publish interval over τ is a/q in lowest terms.

TABLE S2
EACH TESTBED CARRIES A DIFFERENT PART OF THE EVIDENCE. MACHINES, SOFTWARE, AND WHAT RESTS ON EACH TESTBED.

Testbed	Machines	Kernel and software	What rests on it
Cloud	Four Oracle Cloud VM.Standard.E5.Flex instances on a private network: three brokers and one driver.	Ubuntu 22.04, kernel 6.8.0-1057-oracle (EEVDF scheduler), a 1 ms tick; CPython 3.10; chrony-disciplined clocks, logged every minute.	Every measurement the paper does not attribute to the registered campaign or its machines.
Registered campaign	Three Microsoft Azure pairs in one region, each a driver with eight virtual CPUs and a broker with two: two x86 pairs (AMD EPYC 9V74) and one Arm pair (Neoverse N2).	The provider's stock kernels, 6.8.0-1064 and 6.8.0-1065; for the tick block, three kernels built from one source tree that differ in the tick alone, 1, 4 and 10 ms.	The registered campaign (S6), with its tools block, and the later runs that time the receiving thread's wait.

TABLE S3
EVERY CLIENT SETTING THAT CAN MOVE A LATENCY NUMBER, AND WHY IT HOLDS ITS VALUE.

Setting	Value	Why
<i>Kafka producer</i>		
acks	all	Durability matched to Redis's synchronous XADD.
linger.ms	0	No artificial batching delay: on a sparse workload any linger would dominate the quantity measured.
max-inflight	64	At our producer's default of 1 the client blocks on a round trip per message.
compression	none	Football events are small JSON. Compression would add CPU work to the timed path for no representative benefit.
ack-stamp	callback	The default, asymmetric with Redis (S1.3).
<i>Redis producer</i>		
publish workers	pool	Non-blocking dispatch keeps the publish loop on schedule; publish and acknowledgment are timestamped in the worker.
<i>Consumers</i>		
Redis ack-batch	200	Acknowledging each message separately adds a round trip per message.
Redis count	200	Read batch, matched to the acknowledgment batch so neither dominates.
Kafka poll-timeout	1000 ms	The client already amortizes fetches. No per-record round trip exists to remove.
<i>Replay</i>		
speed-up	derived	Plans carry a built-in 120× compression, so the flag is not the rate. The rate is derived from the plan and checked against wall time before every campaign.

thread involved. Under Kafka it is read, by default, in the client's I/O thread when the delivery callback fires. Our Kafka producer's `inline` mode instead timestamps on the calling thread once the publish future resolves, and requires one request outstanding.

The consumer reads t_{recv} when it holds the record, and t_{out} once it has handled it. Our Kafka consumer parses the payload inside `poll()`, before t_{recv} , and our Redis consumer parses it between the two. The processing-time latency $t_{\text{out}} - t_{\text{recv}}$ is therefore 281 ns under Kafka and 19,480 ns under Redis, medians of run medians over 5,913 runs, and negative on none of the 738,730 messages. The parse thus falls inside D for Kafka only, and Redis's median end-to-end latency of 800 μs leaves out a parse 2.4% as long. A longer D puts fewer values of S below zero, and Kafka already has the higher rate, so including that parse would widen the gap between the brokers in Table S4.

Table S4 counts the values below zero in each span over every cloud run that produced matched records, rejected runs included, because a rejected run shows which span falls below zero as well as a clean one does. Only S , the one span whose two timestamps have no causal order (Section IV-A), ever does.

S2. THE SIGN CHECK, AND THE RUNS IT COVERS

The rule is that of Section VI, applied to every run. A run is rejected if more than 1% of its messages carry a negative value in S , in the publish delay $t_{\text{pub}} - t_{\text{sched}}$ or in the processing-time latency $t_{\text{out}} - t_{\text{recv}}$, or if the median of any of the three is negative. A condition is usable only if all of its runs survive. The one-percent line is a convention, and we fixed the rule after earlier campaigns had run, which is weaker than registration.

Over the 884 cloud runs behind our results, the check rejects 459 (51.9%), and 13 of their 40 conditions keep every run. That corpus is not the one of Table S4, which counts every cloud run that produced matched records, rejected or not, 5,913 of them. Of those, 3,976 exceed the one-percent line on S .

TABLE S4

ONLY S FALLS BELOW ZERO, AND UNDER BOTH BROKERS. VALUES BELOW ZERO BY SPAN AND BROKER, OVER THE 5,913 CLOUD RUNS THAT PRODUCED MATCHED RECORDS, ON ONE CLOCK. THE SMALLEST RUN MINIMUM OF THE PUBLISH LATENCY A IS $112\mu\text{s}$, CLEAR OF ZERO.

Span	Origin timestamp	Kafka	Redis	All
Messages		367,894	370,836	738,730
$S = t_{\text{recv}} - t_{\text{ack}}$ share below zero	acknowledgment	31,899 8.67%	30,365 8.19%	62,264 8.43%
$D = t_{\text{recv}} - t_{\text{pub}}$	publish (chain)	0	0	0
$t_{\text{out}} - t_{\text{pub}}$	publish (chain)	0	0	0
$A = t_{\text{ack}} - t_{\text{pub}}$	publish (chain)	0	0	0
$E = t_{\text{out}} - t_{\text{sched}}$	event time	0	0	0

S3. THE MECHANISM AND ITS LAWS IN FULL

Codes beginning E-A name campaigns on the Oracle Cloud testbed, and a code ending in b repeats or extends the one before it. Only the receiving thread's wait (S3.8) was measured elsewhere, on the registered campaign's machines.

S3.1. Real-time priority cuts the rate in every matched pair

Moving the threads that read the clock to real-time priority, `SCHED_FIFO`, cut the negative-span rate between 7- and 80-fold in each of eight matched pairs, with utilization unchanged (Table S5). At real-time priority those threads preempt the background load instead of queueing behind it, so their waiting probability falls while the machine stays as busy. Any account in which utilization alone sets the rate predicts no change.

The pairs come from three campaigns. E-A5 is the original manipulation, E-A5b widened the range of loads to 60 and 95%, and E-A7 replicated E-A5. A rule fixed in advance withholds a pair whose two achieved utilizations differ by more than 0.05. It withheld 0 pairs, and the widest difference in the table, 0.002, is 25 times smaller than the tolerance.

TABLE S5

FACTOR OF 7 TO 80, WITH UTILIZATION MATCHED IN EVERY PAIR. NORMAL-PRIORITY AND REAL-TIME NEGATIVE-SPAN RATES AT THE SAME ACHIEVED UTILIZATION, AND THE FACTOR BETWEEN THEM, FROM THREE CAMPAIGNS. BOTH CONFIGURATIONS' ACHIEVED ρ IS PRINTED SO THAT THE MANIPULATION CAN BE CHECKED PAIR BY PAIR. GENERATED AT BUILD TIME FROM THE CAMPAIGN FILES.

campaign	load	ρ normal	ρ real-time	normal	real-time	factor
E-A5b	60%	0.6055	0.6035	0.0459	0.0017	$27\times$
E-A5b	70%	0.7032	0.7025	0.1032	0.0144	$7\times$
E-A5	75%	0.7531	0.7525	0.1320	0.0034	$39\times$
E-A7	75%	0.7550	0.7537	0.1504	0.0044	$35\times$
E-A5b	88%	0.8806	0.8809	0.2214	0.0034	$66\times$
E-A5	88%	0.8809	0.8812	0.3049	0.0057	$54\times$
E-A7	88%	0.8825	0.8825	0.2526	0.0054	$47\times$
E-A5b	95%	0.9501	0.9501	0.2938	0.0037	$80\times$

Only the normal-priority rate moves with load. The real-time rate stays between 0.0017 and 0.0144 over the whole ladder, as it should if priority moves the waiting probability p of Equation 3 directly. The factor tends to grow with load but not at every step, so it is reported as a range. For E-A5's two pairs its Katz 95% intervals are [21–74] at 75% load ($z = 19.8$) and [33–86] at 88% ($z = 31.9$). They are wide because the real-time configurations hold few values of S below zero. The Wilson intervals on the rates themselves are disjoint in every pair.

E-A7 also registered that real-time priority would cut the thread's waiting probability at least tenfold. Its scheduler counters show the waiting probability falling 1.7- and 2.2-fold at its two loads, and the mean wait 3- to 5-fold, against the falls in the rate that Table S5 gives for E-A7. The prediction failed. Counters that report means cannot see a tail event, so these campaigns leave the factorization of Equation 3 untested. The kernel trace of S3.5 times the wait directly.

S3.2. One utilization, placed two ways, gives two rates

A concentrated load keeps k of the $C = 8$ cores fully busy and leaves the other $C - k$ free, while a spread load keeps every core busy part of the time and leaves none free. If values of S below zero come from a thread that cannot find a free core, the spread load should produce more of them at equal utilization, and it does (Table S6).

At $k = 6$ both placements sit at $\rho = 0.7531$, identical to four decimals, on one machine in one hour, and their rates differ 2.07-fold ($z = 10.3$). No function of ρ can do that, since a function returns one value for one input. The replication E-A6b reproduces the cell at the same ρ and a factor of 2.05, and the direction at $k = 5$. At $k = 7$ the difference is small, since one free core in eight is nearly as useless as none, and only the replication resolves it ($1.19\times$, $z = 3.46$). Multi-server queueing

TABLE S6

AT $k = 6$ ONE UTILIZATION GIVES TWO RATES, IN TWO CAMPAIGNS. CONCENTRATED LOAD LEAVES $C - k$ CORES FREE; SPREAD LOAD DUTY-CYCLES ALL OF THEM AND LEAVES NONE. EVERY CELL IS 2,985 MATCHED EVENTS OVER 25 RUNS, AND z IS A TWO-PROPORTION TEST BETWEEN THE TWO PLACEMENTS OF A ROW. ACHIEVED ρ AT $k = 6$ IS 0.7531 IN ALL FOUR CONFIGURATIONS; AT $k = 5$ IT IS 0.6284/0.6250 AND 0.6284/0.6259, AND AT $k = 7$ 0.8778/0.8809 AND 0.8778/0.8822.

k/C	E-A6 (original)			E-A6b (replication)			factors
	concentrated	spread	z	concentrated	spread	z	
5/8	0.0201	0.0379	4.09	0.0111	0.0509	8.89	1.88 $\times$, 4.61 $\times$
6/8	0.0824	0.1709	10.27	0.0600	0.1229	8.44	2.07 $\times$, 2.05 $\times$
7/8	0.2281	0.2415	1.22	0.1973	0.2342	3.46	1.06 $\times$, 1.19 $\times$

TABLE S7

SLOWER PATH, MORE RELIABLE MEASUREMENT, IN TWO CAMPAIGNS. PAYLOAD PADDING IN TWO CAMPAIGNS. THE MEDIAN OF S SPANS 76.9 $\times$ AND 76.8 $\times$ WHILE THE RATE FALLS 4.09 $\times$ AND 4.26 $\times$ [3.62–5.01], BOTH MONOTONICALLY, WITH UTILIZATION HELD TO THE ρ SPAN SHOWN. EACH CELL IS 2,985 EVENTS.

Padding (B)	E-A10		E-A10b		ρ span
	Median S (ms)	Rate	Median S (ms)	Rate	
0	0.701	0.2603	0.761	0.2338	0.881–0.882
4,096	2.381	0.1906	2.474	0.1652	0.882–0.883
65,536	14.638	0.0888	16.059	0.0784	0.885–0.885
262,144	53.898	0.0637	58.482	0.0549	0.893–0.895

already expects placement to matter at fixed ρ , so the contrast refutes not queueing itself but the single-parameter form, in which ρ alone fixes the rate.

S3.3. A longer path gives fewer values below zero

The same waits must outlast a longer interval to put S below zero, so a longer path should give fewer such values. Padding the payload lengthens the path without touching the scheduler. In E-A10 the median of S rose 76.9-fold over 4 levels while the negative-span rate fell 4.1-fold [Katz 95%: 3.5–4.8], monotonically, and a repeat on another day, E-A10b, agrees (Table S7). Serialization pushes utilization slightly up, which would raise the rate, so the fall is measured against its own confound. An account in which stress on the host causes the values below zero predicts the opposite direction.

S3.4. Load mostly changes how often the thread waits

A thread that reads the clock is either running, and reads it late by a narrow jitter, or waiting for a core. From an idle host to the heaviest load before its processors are fully busy, E-A3's negative-span rate grows 61-fold while the jitter's width grows only 5-fold (Table S8). Load therefore moves mostly the waiting probability p of Equation 3, the chance that the thread is waiting, and little the running state's jitter. A scale family, in which load only widens one distribution, has one parameter and cannot move the two this differently. The tails agree. The two states change p , so the tail curves of two loads should shift vertically and stay parallel on a log scale, where a scale family would rescale them horizontally. Over tail estimates backed by at least 20 events, the log ratio of two conditions' tail masses varies over thresholds by a median factor of 1.34 across conditions, while the scale-family collapse fails by 23 $\times$. That test was exploratory; the registered test of p is the priority manipulation of S3.1.

A wait lasts an interval, so the messages inside one are read late together. A Wald–Wolfowitz runs test on each run's sequence of signs of S finds strong clustering at every load, even with no background load. Coarse kernel timestamps would corrupt messages independently and give $z \approx 0$.

S3.5. A kernel trace predicts the rate unfitted, and changes it

The negative-span rate matches, to within a third, the probability that a wait for a core outlasts the end-to-end latency, measured without reference to S . With `bpfftrace` on the `sched_wakeup` and `sched_switch` tracepoints we recorded the delay between a thread that reads the clock becoming runnable and reaching a core, for every such transition in a run. The two quantities share no probe, no estimator and no data. One is a count of values below zero in application records, the other a kernel histogram of scheduler transitions. Across the three normal-priority configurations of Table S9 the ratio of traced probability to measured rate is 0.78, 1.06, 1.32, with nothing fitted, and the tracer check below withholds one of the three. The ratios lie above and below one, so no bias smaller than a third can be resolved.

The tracer is not a free observer. A first attempt at the traced run, whose probe failed to attach, became an untraced twin. It ran two hours apart with the same design and measured 0.272 against the traced 0.231 ($z = 3.6$). That gap lies within the

TABLE S8

SHORT OF FULL UTILIZATION THE RATE GROWS 61-FOLD AND THE JITTER ONLY 5-FOLD. E-A3, NINE LOAD LEVELS, 25 RUNS EACH, REGISTERED. THE JITTER WIDTH IS $\sigma_{\text{core}} = \text{IQR}/1.349$. VALUES OF S BELOW ZERO CLUSTER AT EVERY LOAD (RUNS-TEST z). THE THREE $\rho = 1$ ROWS ARE DISTINCT OVERSUBSCRIPTION LEVELS THAT THE UTILIZATION CANNOT SEPARATE.

Utilization ρ	Jitter width σ_{core} (ms)	Negative-span rate	Clustering z
0.003	0.126	0.004	-6.8
0.253	0.138	0.004	-6.8
0.504	0.180	0.006	-6.8
0.628	0.181	0.024	-6.9
0.753	0.238	0.076	-5.7
0.878	0.631	0.224	-4.9
1.000	1.224	0.371	-4.3
1.000	1.293	0.317	-4.7
1.000	1.710	0.261	-4.4

TABLE S9

KERNEL TRACING PREDICTS THE NEGATIVE-SPAN RATE, UNFITTED, IN EVERY CONFIGURATION THAT HAS ONE. $\text{Pr}[\text{wait} > 0.5 \text{ ms}]$ FROM THE `SCHED_WAKEUP` AND `SCHED_SWITCH` TRACEPOINTS, AGAINST THE NEGATIVE-SPAN RATE FROM APPLICATION TIMESTAMPS IN THE SAME CELL. EVERY CELL IS 2,985 EVENTS. EVERY TRACED REAL-TIME CONFIGURATION READS EXACTLY ZERO WHERE THE UNTRACED CONTROL READS 15 IN 2,985, SO THAT ZERO IS THE TRACER'S.

Campaign	Configuration	Load	ρ	Traced wakeups	$\text{Pr}[\text{wait} > 0.5 \text{ ms}] / \text{rate}$	Ratio
E-A9	normal	88%	0.8822	551,956	0.181 / 0.231	0.78
E-A9	real-time	88%	0.8819	570,591	0.018 / 0.000	—
E-A9b	normal	75%	0.7543	709,819	0.168 / 0.128	1.32
E-A9b	real-time	75%	0.7531	641,308	0.016 / 0.000	—
E-A9b	normal	88%	0.8825	687,986	0.207 / 0.196	1.06 [†]
E-A9b	real-time	88%	0.8822	649,788	0.016 / 0.000	—
Untraced control: E-A9's first attempt, probe never attached						
—	normal	88%	0.8812	—	— / 0.272	—
—	real-time	88%	0.8809	—	— / 0.005	—

[†]Withheld by the tracer check, 28% drift against a 25% rule fixed in advance: shown, not used.

spread of the untraced normal-priority rates at 88% load in Table S5, and it moves only the rate's level, on which no claim rests. Each prediction is held against the traced run's own rate, so a common perturbation moves both quantities together. A rule fixed in advance, the tracer check, compares a traced configuration only if its rate lies within 25% of its untraced comparator. The replication's 88% configuration drifts 28%, so its comparison is withheld, and its row shows the ratio it would have given.

S3.6. Rival explanations, each excluded or bounded

Five rivals fall to the design and the manipulations. One clock excludes clock skew between hosts. One utilization at rates twofold apart excludes utilization (S3.2). Real-time priority excludes timer noise, which would delay a thread whatever its scheduling class (S3.1), and a longer path that lowers the rate excludes stress on the host (S3.3). Coarse kernel timestamps fall to the runs test of S3.4. Four more need an argument each.

The second thread: Kafka's client reads the clock for the acknowledgment on its I/O thread when the delivery callback fires, and Redis's on the calling thread the moment `XADD` returns (Figure 1). The hand-off is not the mechanism. Redis, which has no second thread, has values of S below zero on 8.19% of its messages against Kafka's 8.67%, and on more of its runs (88% against 65%). Its calling thread, returning from a blocking socket read, must also be woken and given a core before it reads the clock.

The hypervisor: Both testbeds are virtual machines, so steal time, the time a virtual processor is ready to run but waits while the hypervisor runs another machine, is bounded, not excluded. Two observations disagree with steal time as the cause. Real-time priority is a setting inside the virtual machine, which steal time would not yield to, yet it cuts the rate 7- to 80-fold (S3.1). And the upper mode of the traced waits shares its bucket with the base slice that the virtual machine's own kernel derives (S3.7), not with any quantity the hypervisor exposes.

The interpreter lock: The threads that read the clock are CPython, which releases the lock around blocking I/O and retakes it on return, so a timestamp waits for a core and then for the lock. The kernel trace cannot see the second wait, and neither priority nor the size of the waits tells the two apart. The direction of the trace's error settles it. A second wait in series with the run queue would make the trace underestimate the rate, but its ratios, 0.78, 1.06, 1.32, lie on both sides of one. The registered campaign also found values of S below zero in Kafka's own Java client, which has no interpreter lock, at a lower rate (S6.2).

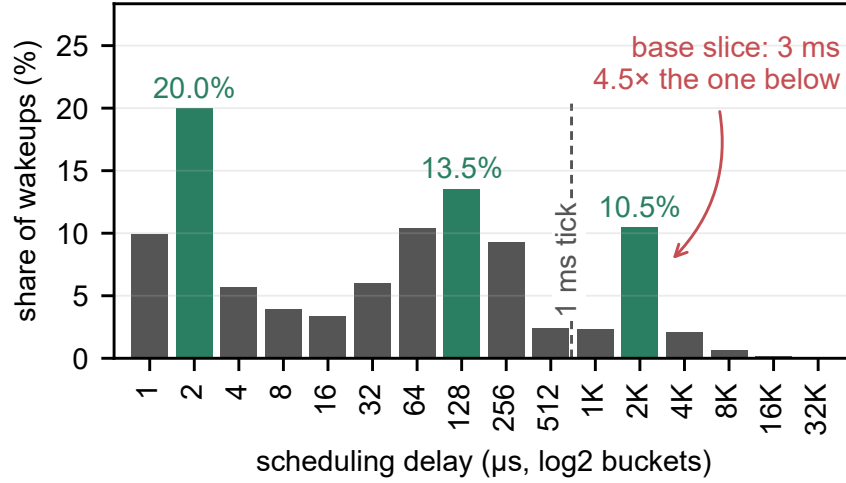


Fig. S1. **Traced scheduling delays are trimodal.** 551,956 wakeups of E-A9’s normal-priority configuration at 88% load (Table S9), `bpfttrace` `log2` buckets, modes shaded. The jitter peaks in the 2–4 μ s bucket, which alone holds 20.0% of all wakeups. A second mode at 128–256 μ s holds 13.5%. The third, at 10.5%, sits in the bucket holding the scheduler’s derived base slice of 3 ms. The count of modes survives a $2\times$ widening of the buckets in both phasings.

The clock: Clocks that disagree across virtual CPUs [S2] are excluded twice. First, the platform probe’s smallest observed increment of `clock_gettime(CLOCK_MONOTONIC)` was 90 ns, an upper bound on the cost of one read. That excludes `acpi_pm` by arithmetic, since its tick is 279 ns, and `hpet` by its published read cost near 1.2 μ s. What remains, `kvm-clock` or `tsc`, carries the coherence guarantee. A `kvm-clock` that fast is on the vDSO path, which `kvmclock.c` enables only when the hypervisor sets `PVCLOCK_TSC_STABLE_BIT`, the flag that guarantees monotonic time across CPUs. A KVM guest selects `tsc` only when the kernel finds it constant, non-stop and stable, and abandons it if it later falls out of step between CPUs. Which of the two ran is unrecorded, and the published read costs vary about twofold, against an order-of-magnitude margin. Second, an offset between clocks large enough to produce the deepest value of S , -99.8 ms, would have driven D far below zero. The smallest D observed, 220 μ s, bounds any such offset 455-fold below what that value needs.

S3.7. The waits’ upper mode survives coarsening, and its bucket holds the derived slice

The traced waits have 3 local maxima (Figure S1), and the upper one survives a change of bucket edges. How many maxima a histogram shows depends on its bin edges as well as on its counts, and these edges came from `bpfttrace`. The per-event waits were not retained, so coarsening is the only test available. Merging adjacent buckets doubles the bin width, and starting the merge at the first bucket or the second puts the edges in different places. The spectrum has 3 maxima on the tool’s own 16 buckets, 3 again under both phasings at 2 times the width, and fewer at 4 times, so the count holds to within a factor of 2 in bin width.

Above the upper mode the counts collapse. They fall 5.0-, 3.4- and 4.7-fold over the next three octaves and then 357-fold, each octave a doubling of the wait, whereas a power law would fall by the same factor in every octave. Power laws fitted below the mode and above it both fail a bootstrap goodness-of-fit ($p < 0.0004$ and $p < 0.0004$, over 2,500 replicates each).

The slice is derived from the kernel’s rule and the campaigns’ own data. Each placement condition loads k cores and records the achieved utilization, so k/ρ recovers 8 cores six times over, with a spread of 0.05. Equation 4 then gives 4×0.75 ms = 3 ms [S3]. The constant is specific to this kernel and became 0.70 ms in v6.15, after our 6.8. The build sets `CONFIG_HZ = 1000`, a 1 ms tick. With run-to-parity, a woken thread that loses the wakeup-preemption check waits for the running thread to finish its slice, and since a task notices its deadline only at the next tick, waits end in a band a tick wide. No scheduler tunable is written anywhere in the archived campaigns, so the derived value is the one that applied. That is strong evidence, short of proof, because the slice was not read off the hosts. The registered campaign later timed the wait itself and found it to be the rest of a slice (S6.3).

S3.8. Timing from the publish call keeps the receiving thread’s wait, under a millisecond on the mean

A latency timed from the publish call carries the receiving thread’s wait for a core. By Equation 1, $D = S + A$ for every message, so the acknowledging thread’s wait cancels: it lengthens A by as much as it shortens S . The receiving thread’s wait does not cancel. The consumer reads its clock only after its thread has been woken and given a core, so every D holds that wait, which is never negative and which the sign check of Section VI cannot see. After the registered campaign we measured it three ways in the same runs, by a plan and four predictions written before any run.

TABLE S10

THREE MEASUREMENTS PUT THE RECEIVING THREAD’S WAIT AT 0.41–0.94 MS OF THE MEAN LATENCY. ONE ROW FOR EACH BROKER AND LOAD. METHOD 1 IS THE FALL IN MEAN D WHEN THE CONSUMER ALONE RUNS AT REAL-TIME PRIORITY. METHODS 2 AND 3 ARE THE FALLS, FROM NORMAL TO REAL-TIME PRIORITY, IN THE MEAN TRACED WAIT AND IN THE MEAN DELAY FROM THE KERNEL’S RECEIVE TIMESTAMP TO THE TIMESTAMP. EACH FIGURE IS A SETUP’S MEDIAN OVER ITS RUNS. THE CLIENT’S WORK IS THE MEDIAN, OVER TRACED RUNS, OF THE DELAY BEYOND THE TRACED WAIT.

Broker, load	Added to the mean of D (ms)			Largest over smallest	D , normal $\rightarrow$ real-time priority (ms)		Client’s work (ms)
	Method 1	Method 2	Method 3		Median	Ninetieth percentile	
Kafka, 75%	0.43	0.70	0.64	1.64	2.03 $\rightarrow$ 1.82	5.21 $\rightarrow$ 4.27	0.37
Kafka, 88%	0.94	0.91	0.92	1.04	2.52 $\rightarrow$ 1.92	5.66 $\rightarrow$ 4.53	0.39
Redis, 75%	0.47	0.50	0.47	1.07	0.89 $\rightarrow$ 0.84	3.06 $\rightarrow$ 0.93	0.11
Redis, 88%	0.43	0.62	0.41	1.49	0.87 $\rightarrow$ 0.83	2.86 $\rightarrow$ 0.89	0.12

The runs used the registered campaign’s first x86 pair (S6.1), both brokers, 75 and 88% load and the base slice at 3 ms, with no delay added to the network. Eight setups, each broker at each load with the consumer at normal or at real-time priority, ran in six shuffled rounds. Three runs added before anything was read gave every setup at least three traced runs, and all 51 runs passed the integrity rule. Method 1 runs the consumer alone in the real-time class at priority 80 and measures how much that lowers D at unchanged load. Method 2 traces the scheduler in a random half of the runs and measures the receiving thread’s own wait for a core before each timestamp, as S3.5 did for the acknowledging thread. Method 3 captures the receiver’s packets in every run with the kernel’s nanosecond timestamps, and times each message from the packet that carries it to the consumer’s timestamp. That delay includes the wake-up, the wait and the client’s own work.

The three estimates run from 0.41 to 0.94 ms (Table S10) and lie within a factor of 1.5 of each other at three of the four points. The wait lives in the tail. At normal priority the median message waited 18–46 μ s and the mean 0.51–0.92 ms, while the ninetieth percentile reached 2.13–2.92 ms. Real-time priority brought the mean wait down to 10–11 μ s. On Redis it cut the ninetieth percentile of D to under 1 ms and left the median almost where it was.

The first prediction held. On both brokers at both loads the wait’s mean at normal priority lay between 0.3 and 1.2 ms and its ninetieth percentile above 1.5 ms, and real-time priority took the mean below 0.1 ms. The other three failed, each on Kafka. There real-time priority lowered the ninetieth percentile of D by less than the 1 ms predicted at 75% load, and moved the median by more than the 0.3 ms allowed at 88%. Method 3’s delay was at least as long as the traced wait for 99.7% or more of every traced run’s messages, against the 95% predicted. But Kafka’s client then worked longer than the 0.3 ms predicted before its timestamp, where Redis’s stayed under it. At 75% load Kafka’s first two estimates lie further apart than the factor of 1.5 allows. We have not tested why Kafka differs.

Three things qualify the figures. The tracer is not a free observer: at normal priority the traced runs’ mean D ran 5–22% above the untraced runs’, so the second method may read the wait somewhat high. In five runs a message arrived more than 150 ms late, three of them during stalls of the whole driver, of the kind the registered campaign marks as paused (S6.2). A setup’s value is a median over its runs, which sets them aside. And three of the four setups at real-time priority hold only two untraced runs, too few to judge the tracer there. The plan, the register of every run and the reading are in the repository under docs/results/recv_wait, read by scripts/recv_wait.py.

S4. THE POSITIVITY FILTER AND THE RETENTION LAW IN FULL

The benchmark’s printed latency describes only the samples its publish schedule lets through the positivity filter, and each link in that claim can be checked against a source line, a counter or a registered prediction. Every benchmark run below is in embedded mode, with the whole benchmark in one JVM.

S4.1. Two source lines make the filter, and counting what they drop changes no printed number

What the positivity filter drops can be counted without changing the filter. In LocalWorker.java:294 the benchmark computes its end-to-end latency as

```
MILLISECONDS.toMicros(now - publishTimestamp)
```

from two millisecond timestamps, so the value, though in microseconds, is a whole number of milliseconds. In WorkerStats.java:95 the positivity filter,

```
if (endToEndLatencyMicros > 0)
```

decides whether that value reaches the histogram. The message has already been counted as received, but nothing counts what the condition drops.

We add counters in an else branch of that condition and change nothing else: not the latency computation, not the filter, not any reported statistic. The benchmark’s output is therefore exactly what it would have been unpatched, and its printed summary can be set beside the retention behind it (Fig. S2b). The four counters, for zero, negative, most negative and kept samples, separate sign. A value below zero and an end-to-end latency shorter than a millisecond both fail a > 0 test, and one

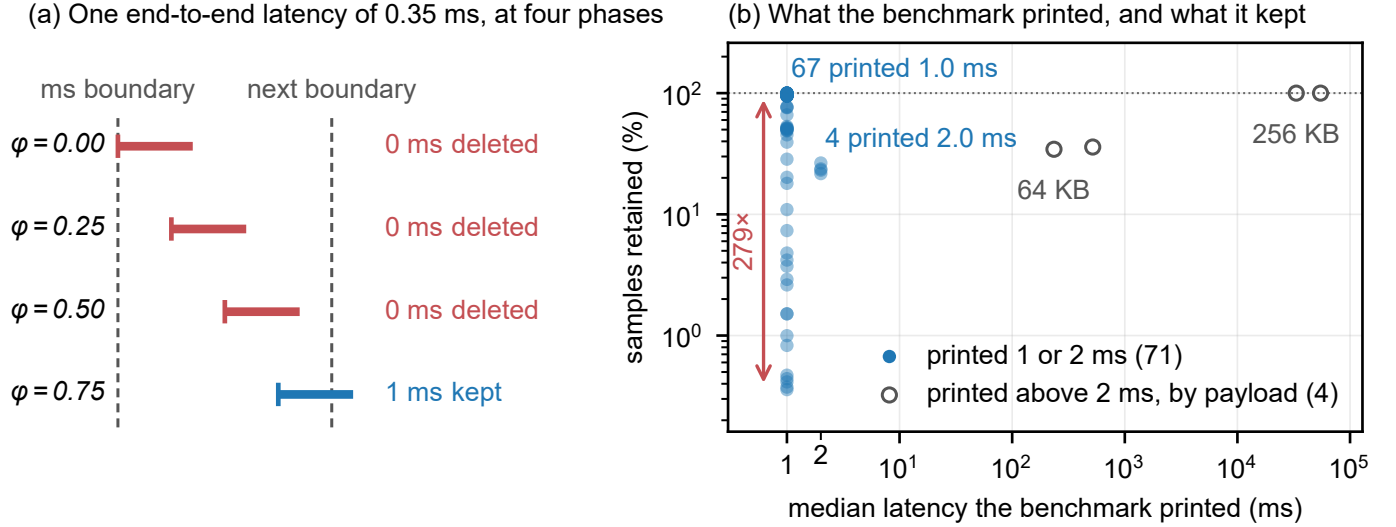


Fig. S2. **Why a sample is deleted, and what the deletion did.** (a) One end-to-end latency of 0.35 ms, starting at four phases within one millisecond. Three finish inside it, compute to 0 ms and are deleted. The fourth crosses a millisecond boundary, one of the *grid* instants at which the clock’s digit changes, and is kept, as exactly 1 ms. (b) Each of the 75 settings with a captured summary, at the median the benchmark printed and its retention. 71 settings print 1.0 or 2.0 ms while retention spans 279 $\times$. Above the grid, nothing is lost only at 256 KB.

unsigned total cannot tell them apart (S4.4). The patch is `external/omb/workerstats_discard_counter.patch` in the artifact.

The counts are exact. The counters are shared across the JVM and their totals printed once at shutdown. A run that fails discards nothing, and its zero count would look measured, so a validity gate writes it as invalid, with its reason and no count. Unlike the benchmark’s histograms, our counters include the one-minute warmup, yet a matched sweep with the warmup disabled leaves retention unchanged. The campaign totals 223 instrumented runs, swept one axis at a time with background load, message size, producer rate and warmup otherwise fixed.

S4.2. The phase of the publish schedule sets retention, as a 1970 counter note already showed

Where each publish instant falls within a step of the clock decides whether its sample survives. An end-to-end latency shorter than the step τ survives only if it crosses a grid instant, where the timestamp changes value. Otherwise both timestamps round down, or floor, to the same value, and the positivity filter deletes the zero difference (Fig. S2a). When phases are uniform, retention is $\theta = T_{\text{true}}/\tau$, which is Equation 5 of the paper.

A paced producer does not draw its phases uniformly. If its publish interval T_{pub} satisfies $T_{\text{pub}}/\tau = a/q$ in lowest terms, it visits only q distinct phases, so its retention is a whole count out of q . Call the retentions $0, 1/q, \dots, 1$ the vertices of the configuration’s q -lattice, and the interval between neighboring vertices, $100/q$ percentage points wide, a lattice step. At an incommensurate rate, whose ratio has no small denominator, the vertices are dense and retention is θ . Elsewhere, replicates of one configuration land on the vertices that bracket θ , and their spread obeys

$$\text{spread} \longrightarrow \begin{cases} 100/q & \text{when } \theta \text{ sits mid-step,} \\ 0 & \text{when } \theta \text{ sits on a vertex.} \end{cases} \quad (\text{S1})$$

The step width is thus an upper bound on the spread, reached only when replicates realize both bracketing vertices. Retention at the incommensurate rates puts T_{true} at 0.49 ms, half a millisecond to within replicate noise (Section V-C). One half is a vertex of every q -lattice with even q and of none with odd q , so the spread rule predicts full step width for odd denominators and almost none for even ones.

Hewlett-Packard’s Application Note 162-1 [S4] carries this arithmetic for a counter timing an interval. For an interval of $Q + F$ clock periods it gives $\Pr[R = Q+1] = F$, which is Equation 5, and the denominator of the ratio of repetition rate to clock rate in lowest terms, our q , fixes how many distinct readings the counter can produce, so averaging N readings improves resolution by at most a factor $1/q$, not $1/\sqrt{N}$. Its cure, phase jitter on the repetition rate, becomes for a benchmark randomized publish instants (the paper’s Table I).

S4.3. Retention keeps to its lattice, and a registered change of message size moved one configuration onto a vertex and off it

The phase account predicts each configuration’s behavior from its denominator alone. Its starkest case is $q = 1$: where the publish interval is a whole number of milliseconds, every publish has the same phase, and replicates of one configuration

TABLE S11

REPLICATE SPREAD FOLLOWS THE PHASE DENOMINATOR q IN 10 OF THE 12 COMMENSURATE CONFIGURATIONS. PAYLOAD, LOAD, DURATION AND HOST ARE FIXED. THE POSITION IS THE DISTANCE FROM THE MEASURED CONTINUOUS VALUE TO THE NEAREST VERTEX, AGAINST HALF THE STEP WIDTH. A CONFIGURATION IS PREDICTED FULL WHERE ITS POSITION EXCEEDS A HALF, AND SHOWS FULL WHERE ITS SPREAD EXCEEDS HALF THE STEP WIDTH. THE DAGGERED ROWS ARE THE TWO MISSES, RESOLVED IN TABLE S12. INCOMMENSURATE CONFIGURATIONS HAVE NO LATTICE STEP. THEIR SPREADS OF ABOUT THREE POINTS ARE THE NOISE FLOOR.

rate	T_{pub}/τ	q	n	width	pos.	predicts	spread	shows
1000/s	1/1	1	5	100.0	0.98	full	99.4	full
500/s	2/1	1	5	100.0	0.98	full	99.5	full
250/s	4/1	1	5	100.0	0.98	full	98.7	full
400/s	5/2	2	5	50.0	0.04	flat	17.6	flat
600/s	5/3	3	6	33.3	0.95	full	32.7	full
300/s	10/3	3	10	33.3	0.95	full	31.2	full
800/s	5/4	4	5	25.0	0.07	flat	7.2	flat
1250/s [†]	4/5	5	5	20.0	0.91	full	7.6	flat
625/s	8/5	5	10	20.0	0.91	full	19.3	full
875/s	8/7	7	5	14.3	0.87	full	10.7	full
700/s [†]	10/7	7	5	14.3	0.87	full	6.5	flat
900/s	10/9	9	5	11.1	0.84	full	6.9	full
<i>incommensurate — phase set effectively continuous</i>								
333/s	1000/333	>64	5	—	—	≈ 0	4.5	—
383/s	1000/383	>64	5	—	—	≈ 0	2.8	—
457/s	1000/457	>64	5	—	—	≈ 0	2.1	—
611/s	1000/611	>64	5	—	—	≈ 0	3.1	—
889/s	1000/889	>64	5	—	—	≈ 0	1.7	—
1053/s	1000/1053	>64	4	—	—	≈ 0	3.0	—
1219/s	1000/1219	>64	4	—	—	≈ 0	4.5	—

TABLE S12

RETENTION SITS ON THE q -LATTICE IN NINE OF THE TEN POWERED CONFIGURATIONS, TESTED AGAINST A CONTINUUM NULL. HERE $\bar{d}$ IS THE MEAN NORMALIZED DISTANCE TO THE NEAREST VERTEX, AND THE NULL COLUMN IS ITS MEAN UNDER THE SIMULATED NULL. THE COLUMN p_{Holm} IS A MONTE CARLO p -VALUE (4,000 DRAWS), HOLM-CORRECTED WITHIN ALL 12 CONFIGURATIONS; IT DECIDES THE VERDICT. FLAT (COINCIDENT) MARKS CONFIGURATIONS WHERE THE TWO HYPOTHESES PREDICT ALIKE AND THE TEST HAS NO POWER.

rate	a/q	n	$\bar{d}$	null	p_{Holm}	verdict
1250/s	4/5	5	0.131	0.351	0.003	grid
1000/s	1/1	5	0.003	0.479	0.003	grid
900/s	10/9	5	0.265	0.340	0.131	not resolved
875/s	8/7	5	0.209	0.382	0.003	grid
800/s	5/4	5	0.104	0.058	1.000	flat (coincident)
700/s	10/7	5	0.329	0.416	0.009	grid
625/s	8/5	10	0.112	0.447	0.003	grid
600/s	5/3	6	0.034	0.469	0.003	grid
500/s	2/1	5	0.043	0.490	0.003	grid
400/s	5/2	5	0.093	0.020	1.000	flat (coincident)
300/s	10/3	10	0.221	0.468	0.003	grid
250/s	4/1	5	0.181	0.489	0.003	grid

spread over almost the whole range, while those at incommensurate rates agree to within a few points (Table S11). The spread rule of Equation S1 holds in 10 of the 12 commensurate configurations. A spread above half the step width counts as full, so the test needs no tolerance. The two misses, 700 and 1250 msg/s, are predicted full and measured flat. Spread is a range over replicates that can each land on only two vertices, so a configuration shows its full width only if its replicates land on both. At $q = 5$ and $q = 7$ the upper vertex is the less likely of the two, so five replicates can all fall on the lower one even when the lattice holds exactly, and spread cannot tell that outcome from a refutation.

For that reason we built a second test after the campaign had run. It was not registered, and its first results were committed in the same change as its code. For each configuration, $\bar{d}$ is the mean distance of its replicates to the nearest vertex of its q -lattice, divided by the step width. The null is a continuum, in which replicates are drawn from a Normal distribution centered on the retention the incommensurate rates imply at the configuration's rate, with a measured standard deviation. Neither parameter is fitted to the configuration under test. The one-sided p -value comes from 4,000 Monte Carlo draws at the observed number of replicates, and is Holm-corrected across all 12 configurations. Of these, ten are powered, meaning the null and the lattice predict different distances, and nine of those reject the continuum at $\alpha = 0.05$ after correction (Table S12), among them both configurations the spread rule missed, 700 msg/s at 0.009 and 1250 msg/s at 0.003. The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$), so we report it as unresolved. The two unpowered configurations, 400 and 800 msg/s, have lattices that pass through θ , where the two hypotheses coincide.

The registered test that manipulates the physics is the payload flip. Payload moves T_{true} , and so moves θ within its lattice

TABLE S13

AT A RATE INCOMMENSURATE WITH THE MILLISECOND, RETENTION READS OFF THE END-TO-END LATENCY. RETENTION AGAINST PAYLOAD AT 457 MSG/S, WHERE SAMPLES STAY DEPHASED. THE IMPLIED T_{true} INVERTS EQUATION 5. UP TO 8 KB THE ADDED DELAY IS COMPARABLE TO THE REPLICATE NOISE; THE 32 AND 64 KB LEVELS CARRY THE RESULT.

Payload	Retention (3 reps)	Median	Implied T_{true}
200 B	51.08, 52.03, 52.88	52.03%	0.520 ms
1 KB	50.11, 50.32, 51.47	50.32%	0.503 ms
2 KB	52.13, 54.62, 54.77	54.62%	0.546 ms
4 KB	51.86, 54.17, 56.24	54.17%	0.542 ms
8 KB	54.92, 55.22, 57.89	55.22%	0.552 ms
32 KB	68.01, 68.44, 69.40	68.44%	0.684 ms
64 KB	84.21, 85.36, 86.82	85.36%	0.854 ms

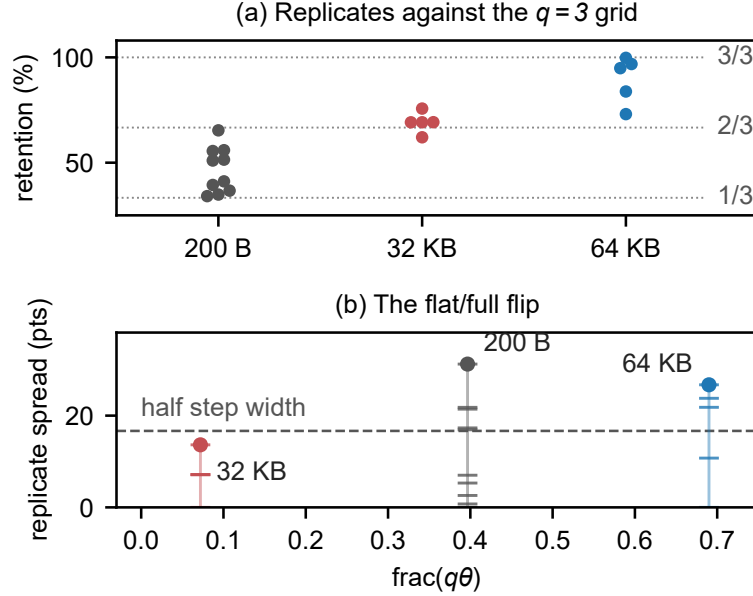


Fig. S3. **One manipulated variable moves a configuration onto its lattice and off it.** (a) Replicates at 300 msg/s ($q = 3$) by payload, against the thirds lattice: 200 B straddles both branches, 32 KB pins near the 2/3 vertex, and 64 KB re-opens to the upper step. (b) Replicate spread against $\text{frac}(q\theta)$, each stick showing the replicates its spread is made of, with θ each configuration's own mean retention in this campaign. The registered prediction placed 32 and 64 KB at 0.055 and 0.56 from the payload sweep's θ . The flat–full boundary at half the step width is crossed in both directions.

step, with nothing else changed. Measured at an incommensurate rate (Table S13), 32 KB puts θ almost on the 2/3 vertex of the $q = 3$ lattice and 64 KB puts it mid-step. The prediction was that this one manipulation would move a 300 msg/s configuration onto a vertex and off it again.

The spread flipped from full to flat and back, as predicted (Fig. S3). At 32 KB the five replicates spanned 62.06–75.69%, a spread of 13.6 points, and at 64 KB they spanned 73.06–99.78%, a spread of 26.7 points. Both cross the flat–full boundary at half the step width, 16.7 points, in the predicted direction. One registered detail failed: the 32 KB configuration pins at 69.2%, not on the vertex at 66.67%. The flip was one of six predictions registered with falsifiers before its campaign. It was confirmed, three were refuted by their falsifiers, one could not be evaluated, and one split (postmortem, S1.7).

S4.4. Two floored clocks disagree by exactly one millisecond, so the sign of a discard names the failed clock

The sign of a discarded sample says which clock failed, and the benchmark's two drivers show both kinds. The Kafka driver's 214 runs discarded 10,913,263 samples, and not one was negative. The Redis-driver replication's 9 runs carry all 41,403 of the campaign's negatives, every one exactly $-1000 \mu\text{s}$.

The drivers explain the difference. The Redis driver takes its publish timestamp from `entry.getID().getTime()` in `RedisBenchmarkConsumer.java:72`, the stream-entry identifier the Redis server assigns on XADD. The Kafka driver uses a producer-set timestamp. So the Redis span subtracts a server-side timestamp from a consumer-side clock reading, across two hosts, with both floored to the millisecond, while the Kafka span floors two timestamps whose difference is taken inside one JVM.

Two clocks floored to the same resolution and offset by less than one millisecond can disagree by one millisecond and by no other amount. The only negative a floored cross-host difference can produce is therefore $-1000 \mu\text{s}$, and every one of the

TABLE S14

TEN TOOLS, READ AT SOURCE. THE SPAN EACH MEASURES, THE CLOCK IT READS, WHAT HAPPENS TO A VALUE AT OR BELOW ZERO, AND WHETHER THE TOOL COUNTS WHAT IT DISCARDS; ONE ROW PER SPAN WHERE A TOOL MEASURES TWO, AND A DASH WHERE NOTHING IS DISCARDED. THE SOURCE LINE BEHIND EVERY ROW IS IN THE ARTIFACT’S REGISTRY.

Tool	Span measured	Clock	What happens to the value	Counts
Apache Kafka (bundled)	end-to-end	nanosecond	truncate to the resolution	no
Apache Kafka (bundled)	publish	millisecond	truncate to the resolution	no
Apache Pulsar perf	end-to-end	millisecond	filter, ≥ 0	no
NATS CLI	end-to-end	nanosecond	refused by own library	no
OpenMessaging Bench.	end-to-end	millisecond	filter, > 0 , truncate to the resolution	no
OpenMessaging Bench.	publish	nanosecond	—	—
RabbitMQ PerfTest	end-to-end	nanosecond	substitute a value	no
Rezulus	scheduler wakeup	nanosecond	discard	yes
blktrace/btt	block stage	nanosecond	substitute a value	no
emqtt-bench	end-to-end	millisecond	—	—
fio	I/O completion	nanosecond	substitute a value	no
wrk2	end-to-end	microsecond	refused by own library	no

41,403 negatives is that value. Clock synchronization was clean at every logged minute, so this failure needs no misbehaving clock, only two clocks whose readings are floored. A difference taken inside one process has no second clock to disagree with, which is why the Kafka discards hold no negative. The same positivity filter thus hides a loss of resolution under Kafka and a clock failure under Redis, and an unsigned discard counter could not tell them apart.

S4.5. An independent harness reproduces the deletion, and the benchmark’s distributed mode never completed a run

The deletion does not depend on the benchmark’s code. We reproduced the positivity filter in a Python harness that shares no code with the benchmark, with the same timestamp, subtraction and filter. On one clock it recorded 0 negative differences across 905,040 samples in 12 runs, as a causal chain requires, and the grid reappeared intact. Across two hosts with disciplined clocks, 12 runs again recorded 0 negatives in 905,040 samples and an intact grid, but the deleted fraction followed the drift of the clock offset. Of the three rates, only the incommensurate one, 457 msg/s, has no lattice spread wide enough to hide the drift. Over four replicates there, cross-host retention wandered from 13.4 to 26.9%, while on one clock the same configuration’s replicates spanned 0.98 points.

The benchmark’s own distributed mode, where its subtraction spans two clocks, could not be measured. It never completed a run in eleven attempts: eight at three background-load levels, including none, and three more run with full diagnostics. The unloaded attempts failed like the loaded ones, which refutes our own hypothesis that processor starvation was the cause, and the three diagnosed attempts failed identically, in the coordinator-to-worker path before any workload started. The first attempt, made before the validity gate of S4.1 existed, wrote a vacuous zero that the artifact keeps marked invalid, and the gate refused every later attempt with its reason. The distributed mode’s exposure to the deletion therefore rests on reading its source. The timestamp is written on the producer’s host and read on the consumer’s, and the positivity filter drops every difference at or below zero.

S5. TOOLS READ AT SOURCE, TOOLS RUN, AND PUBLIC REPORTS

Tools read at source. Seven of the ten tools we read at source dispose of a value at or below zero without counting it. Every verdict traces to a line of source a reader can check. Each row of the registry file, `docs/results/external/harness_registry.csv`, carries one source line, the file and symbol it came from, the upstream URL and the date it was read. The class labels are not stored. They are recomputed on every build and every test run by the classifier that read the OpenMessaging Benchmark a month before the other tools were audited (`classify()` in `scripts/audit_external_harness.py`), so a reader who disagrees with a verdict can check the line against the pattern that matched it. Its patterns were written against JVM sources and had to be extended before they could read an Erlang guard or clock call, and they see only what they recognize, so the registry’s count of disposals is a lower bound for the field.

We call a tool’s own conditional on the sign of an interval its *guard*. A guard is a deletion only if what it guards is the sample itself, not a counter. Table S14 is the registry, one row per span because one tool can treat two spans differently, generated from the registry file at build time. Its sources, in row order, are [S5], [S5]–[S8], [S8]–[S14].

The seven silent disposals take three forms. A filter drops what its comparison excludes and reports no count of what it removed. The test > 0 deletes every sample that computes to exactly zero, while ≥ 0 keeps those and drops only values below zero. The benchmark filters at > 0 . Apache Pulsar’s performance client guards at ≥ 0 , so the zeros the benchmark deletes pile up visibly at zero. A substitution keeps the sample count, so any retention computed downstream reads 100%, which makes it harder to detect than a filter. A library refusal leaves the sample unrecorded because the tool never checks whether its library rejected it. Truncation is a separate loss: it keeps every sample but loses precision. The benchmark truncates before it filters, and Kafka’s bundled tools only truncate. The notes below carry what the table cannot.

TABLE S15

ELEVEN TOOLS, RUN. THE REGISTERED CAMPAIGN’S TOOLS BLOCK, FOUR ROUNDS PER TOOL ON ONE PAIR OF MACHINES. INTERVAL: WHAT THE TOOL TIMES, AS ITS INVOCATION WAS READ BEFORE ANY RUN. SLOPE: HOW FAR ITS FIGURE MOVED PER MILLISECOND OF DELAY, DIVIDED BY THE NUMBER OF TIMES THE INTERVAL CROSSES THE DELAY, IN ITS LOWEST AND HIGHEST ROUND. CLOCKS: HOW MANY CLOCKS ITS TWO TIMESTAMPS COME FROM, AS MOVING THE TOOL’S CLOCK BACK BY MORE THAN THE INTERVAL IT TIMES SHOWED. RUNS: THE OFFSET RUNS THAT DECIDED THIS, OF ALL OF THAT TOOL’S. THE REST WERE UNDECIDED.

Tool	Interval	Slope	Clocks	Runs
hey	round trip	1.00–1.04	one	8/8
k6	round trip	1.00–1.02	one	8/8
mentier	round trip	0.96–1.00	one	7/8
Valkey benchmark	round trip	0.98–1.05	one	8/8
Vegeta	round trip	0.99–1.00	one	8/8
wrk2	round trip	0.92–1.04	one	4/8
Kafka producer	to ack.	1.00–1.14	one	8/8
Kafka end-to-end	one process	1.01–1.02	one	6/8
NATS latency	one process	0.99–1.06	one	7/8
RabbitMQ PerfTest	one process	0.97–1.00	one	6/8
librdkafka perf.	two processes	0.96–1.05	two	6/8

- The OpenMessaging Benchmark appears twice because it treats its two spans differently. It times its end-to-end latency on a millisecond clock and filters it, and times its publish latency in nanoseconds and records every sample (Section V-C).
- The EndToEndLatency tool bundled with Kafka cannot produce a value below zero, because it uses one thread, one monotonic clock and a synchronous publish call. It loses the measurement anyway. Each sample prints in floating point, but the `long[]` its percentiles come from is filled by the integer division `elapsed / 1000 / 1000`, so on a path faster than a millisecond its p50, p99 and p999 all read zero. The companion ProducerPerformance takes its publish-to-acknowledgment difference on `System.currentTimeMillis()` and casts it to `int`.
- In `fio` the guard appears in three timing functions, each returning zero for a negative interval under the comment “time warp bug on some kernels?”. Nothing records that it fired.
- In `btt`, `tdelta()` returns one nanosecond whenever the stage timestamps are out of order, with no comment, no counter and no mention in the manual page.
- The `wrk2` load generator detects the negative interval, prints a block that begins “We are about to crash and die”, and records the value anyway into its own copy of `hdr_histogram.c`. That copy’s `assert()`s are live in the shipped build, and it returns `false` for an out-of-range index, a refusal the tool never checks. Either the assertion aborts the run or the refusal drops the sample, and nothing counts the loss.
- In `emqtt-bench` the `> 0` guard protects a counter and leaves the histogram alone, so nothing is deleted from the sample.
- RabbitMQ PerfTest hands its difference on with no sign test, but the method that computes it, `getDifference`, returns its absolute value, so a negative span enters its metrics as a positive one of the same size, uncounted.
- The NATS CLI appends every difference, negatives included, to the list its minimum, median and maximum are read from, but records its percentile table into `hdrhistogram-go`, whose bound check refuses the index a negative value maps to. The CLI ignores the error, so its percentile table disagrees with its minimum, median and maximum, and nothing counts the difference.
- Rezolus counts its discards in a metric that names the cause.
- Two further tools we read are not in the count, because they do not make the measurement. SPECjms2007 defines Delivery Time and never reports it, and Redpanda’s `rpk` ships no latency benchmark.

Tools run. Reading a tool tells what it does with a value below zero; running it tells whether it ever meets one. The registered law campaign ran eleven benchmarking tools on the first x86 pair, four rounds each, under predictions fixed from their source before any run (its tools block, frozen with the plan; every amendment that touched it is listed in `freezes/README.md`). Table S15 lists them. Under a delay added in steps, every tool’s slope lies in the band fixed in advance, and ten tools were judged to time on one clock in every run that decided it. The one tool spanning two clocks kept every value, negatives included, in each round that decided it. The per-run verdicts are in `docs/results/tools/tools_t1.csv` and `tools_t2.csv`, folded by `scripts/tools_block.py` from the runs, and the write-up with its five recorded faults is `docs/results/tools/t1-t4-eleven-tools.md`.

Public reports. Published results rarely carry the deletion’s signature, and no report we found says how many samples it kept. The audit behind that sentence was registered, committed and public before any report was read (`freezes/29-published-results-audit`). It fixed the signature, the places to search, the rule for inclusion, the coding and the outcomes, and nothing in it changed afterwards.

The primary signature is a printed end-to-end median of exactly 1 ms, the smallest value the benchmark can print and the value every retained sample takes when the true end-to-end latency is shorter than the resolution. The strong signature adds a

p99 of exactly 1 ms. Neither says a published number is wrong. Each says the number came from a fraction of the samples, a fraction the report does not state.

The search covered Google Scholar, Semantic Scholar, the web, the engineering blogs of the vendors whose brokers the benchmark has drivers for, and the benchmark’s repository and its public copies, for reports dated up to 28 September 2026. A report was included if it named the benchmark or a fork of it as its tool and printed at least one numeric end-to-end percentile.

The audit included 43 reports, printing 1,100 configurations, of which 868 print a median. The primary signature occurs in eight configurations in two reports, both public repository text. Five are Redis rates in a notebook behind a 2023 journal paper that shows its medians only in a chart, and three are maximum-rate runs in three result files of a 2021 fork. The strong signature occurs nowhere, and no included report states its retention or a discard count.

The rules did not foresee several patterns. 14 of the included reports print medians below a millisecond, which the original benchmark’s clock cannot produce, among them vendors’ copies whose maintainers changed how the span is timed. One 2024 fork prints p95, p99 and p999 all at exactly 1 ms beside an average just above it and no median column, so by the registered rule it does not count. Four configurations print a median of zero, from empty histograms. The coding, the search log and every excluded report with its reason are in `docs/results/external/published_results_audit` in the artifact (postmortem, S25.1).

S6. THE REGISTERED LAW CAMPAIGN

A law for the negative-span rate, fixed before its runs, had one prediction confirmed on every campaign that tested it, three confirmed on one broker only, and the rest not confirmed (Table S16). The one confirmed everywhere is that load raises the plateau. Neither prediction that the fall follows a slice set by hand was confirmed, the predictions of its width held on one broker at most, and Python’s plateau exceeded the registered limit relative to Java’s (P8). Frozen programs judged every prediction over 8,102 runs on machines the earlier campaigns never ran on.

S6.1. The law, and the design that tested it

The registered law says that the scheduler’s settings alone fix where the negative-span rate falls as the end-to-end latency lengthens, and how wide the fall is. It assumes that a thread kept waiting waits out a whole slice s of another task, and then up to one tick h . The rate then sits on a plateau $c + p$ while the end-to-end latency D is shorter than the slice, falls across one tick, and rests on a floor c beyond it:

$$r(D) = c + p \cdot \begin{cases} 1, & D \leq s, \\ (s + h - D)/h, & s < D < s + h, \\ 0, & D \geq s + h. \end{cases} \quad (\text{S2})$$

Here p is the waiting probability of Equation 3, and c is the floor that ordinary short delays leave. The slice and the tick predict where the fall starts and how wide it is; the plateau’s height and the floor are measured. In the plan, a reading is negative when the thread that reads the clock for the acknowledgment waits longer than the margin $D - g$, a run’s median end-to-end latency less its median publish latency g , the plan’s letter for A .

Three pairs of Microsoft Azure machines in one region ran the campaign, each a driver with eight virtual CPUs and a broker with two; two pairs have AMD EPYC 9V74 processors and one has Arm Neoverse N2. The plan’s blocks A1 to A9 are written L1 to L9 here. L1 sets the slice on the first x86 pair, and L4 on the Arm pair. L2 sets the tick, on three kernels built from one source tree that differ in the tick alone: 1, 4 and 10 ms. L3 sets the load, L5 the number of online CPUs, L7 real-time priority for the threads that read the clock, and L8 the client, Kafka’s own Java client against our Python one. L6 records every `python3` thread’s waits in half of L1’s and L3’s runs, and L9 records each of those threads’ scheduling events in half its own runs. Each block carries numbered predictions, stated and judged in Table S16. A further block, M0, asked a question about bursty traffic that bears on none of the claims here.

Each run replays messages at a steady 50 a second and measures its end-to-end latency, publish latency and negative-span rate after a 30 s warm-up. The end-to-end latency is set by delaying only the traffic to the receiver’s own network address, so the acknowledgment’s path is untouched. Each session first calibrates how far each added delay moves the end-to-end latency, and an end-to-end latency shorter than the client’s own with nothing added cannot be set, and was not run. The integrity rule judges every run as it ends. A run counts when every condition it was meant to have took effect, from the load within 3 points of its target to the added delay within $50 \mu\text{s}$ of its setting. Otherwise it is repeated, up to three attempts. A stopping rule on the publish latency stopped a campaign whose median publish latency moved too far, until the final version made it only record. The plan’s own name for the publish latency, and for the stopping rule on it, is “got-it”.

A round runs every setup once. Each campaign that tests a prediction took its rounds, at least four and at most forty, from simulating its own decision rule on data that follow the law. L2 and L9, which have no rule the simulation can take, ran at the floor of four, and no campaign ran more than 24. A prediction about where the fall sits is read twice, on a free reading, a curve required only never to rise, and on the law’s fitted shape, and it passes only where both agree. The judging read 8,102 runs in 156 campaigns, taken from 16 to 27 September 2026, and 7,996 of them counted.

The plan was first frozen as version 5 on 16 September 2026, before its first run, and the version in force at the end is the thirty-second. A frozen version is never edited. A change is a new, numbered freeze, public before the runs it covers, whose first section says what changed, why, and whether a result already seen prompted it. Three kinds of change followed the campaigns' own results. Version 14, after L3's first campaign could not place the fall at 50% load, judged P3a only where the plateau reaches 2%. The stopping rule was corrected five times, each time after it had stopped a campaign. And three versions corrected judging programs that had already judged, each stating what the verdicts it touches had been. Blocks added as the campaign ran, in versions 24 and 27 to 29, were each read by a program fixed before their runs, and P6 was given its program in version 30, at a margin no run had been read against. Every freeze is in the repository's `freezes/` folder, whose index says what each one changed and whether a result prompted it.

S6.2. Load raised the plateau everywhere, and the fall did not sit where the law put it

Load raised the plateau in all five load campaigns, all on Kafka, with every rise's interval above zero. Table S16 gives each verdict as its frozen program returned it. On Kafka, real-time priority took the plateau to zero in every run, and the fall followed the core count at the one count whose slice is at least Kafka's shortest end-to-end latency. Kafka's own Java client shows the same fall and the same effect of priority, at a plateau a third to a quarter of the Python client's. The Arm pair has values of S below zero as the x86 pairs do (Figure S4).

On the first x86 pair the free reading placed the fall's halfway point inside its band at every slice each broker reaches, four for Kafka and five for Redis. The fitted shape did so at two of four slices for Kafka and two of five for Redis. On the Arm pair the free reading held on both brokers and the fitted shape failed on both. P1 and P9 need both readings, so neither is confirmed. Load moved the fall toward shorter end-to-end latencies in every campaign, and P3a held in none.

TABLE S16

ONE PREDICTION CONFIRMED ON EVERY CAMPAIGN, THREE ON ONE BROKER, AND THE REST NOT. EVERY PREDICTION REGISTERED FOR BLOCKS L1 TO L9, THE NUMBER IT WAS JUDGED ON (ALL RUNS; A SLOPE, RATIO OR MOVE WITH ITS 95% INTERVAL, FROM RESAMPLING WHOLE ROUNDS), AND ITS VERDICT, BROKER BY BROKER WHERE THE TWO DIFFER. A PREDICTION JUDGED BROKER BY BROKER IS CONFIRMED ONLY WHERE IT HOLDS ON BOTH.

	Block, pair	The prediction	On all runs	Verdict
P1	L1, first x86	the fall's halfway point inside $[s, s + h]$ at all but one of the slices a pair reaches, at least four of them; its slope on s in 0.8–1.2	slope, free and fitted: Kafka 0.758 (0.596–1.023) and 0.230; Redis 1.007 (0.844–1.169) and 0.662	not confirmed
P9	L4, Arm	on the Arm pair, inside at every slice it reaches, at least two; the same slope	Kafka 0.830 (0.627–1.172) and 0.285; Redis 1.153 (0.753–1.325) and 0.798	not confirmed
P2	L2	the fall 2.5 to 5.5 times wider at a 4 ms tick than at 1 ms	ratio, free and fitted: Kafka 1.784 and 1.818; Redis 1.064 and 1.126 (S6.3)	not confirmed
P2b	L2	6 to 14 times wider at a 10 ms tick	Kafka 4.786 and 5.358; Redis 3.045 and 3.339 (S6.3)	not confirmed
P2c	L2	the width on the tick a line through zero, slope 0.8–1.2	fitted slope: Kafka 0.968 (0.271–1.125); Redis 0.874 (0.807–1.034)	confirmed on Redis only
P2d	L2	the fitted start within 0.25 ms of the slice at every tick	within it at one of twelve slice and tick pairs	not confirmed
P3a	L3, five Kafka campaigns	the halfway point moves less than 0.25 ms from 75% to 88% load	free move: $-0.009, -0.188, -0.086, -0.011$ and -0.309 ms	not confirmed on any
P3b	L3, five Kafka campaigns	load raises the plateau	rise, free: 0.048 (0.044–0.052), 0.047, 0.048, 0.041 and 0.050	confirmed on all five
P4	L7, first x86	real-time priority cuts the plateau at least five times wherever it is at least 2%	Kafka 0.0405 to zero; Redis's plateau 0.0069 (S6.3)	confirmed on Kafka only
P6	L6	a recording of every thread's waits predicts the rate within 0.67–1.5	median ratio 2.49 to 13.08 over four pair and broker parts; no setup inside	not confirmed
P7	L5, second x86	the default slice follows the core count, and the fall follows it	slices read back at 2, 4 and 8 CPUs as the rule gives; the fall on Kafka at 8 CPUs, Redis's fitted shape missing at all three	confirmed on Kafka only
P8	L8, first x86 and Arm	Java has the same fall and priority effect, and Python's plateau is at most 1.5 times Java's	Python over Java: 3.251 (2.795–3.907), and 4.253 (4.013–4.560) on Arm	not confirmed
L9-1	L9, three pairs	at least 90% of negative readings mostly spent waiting for a CPU	79.4 to 92.0%	not confirmed
L9-2	L9	the time-weighted waits predict the rate within 0.67–1.5	median ratio 0.104 to 0.165	not confirmed
L9-2b	L9	the share of acknowledgments whose own wait outlasted $D - g$ predicts it	inside the rule on four of six parts; median ratio 1.11 to 1.30	not confirmed

The plan keeps every run, and re-runs none whose conditions held. Void runs, runs the integrity rule did not count, and the 18 runs of six false starts, each false start replaced at once by a campaign of the same design, are judged by no program. Four marks leave a run counted: flagged, paused, far from its setup's other repeats, and traced. The stopping rule would have stopped the 14 flagged runs, taken while it only recorded. In the 222 paused runs a message arrived more than 150 ms late,

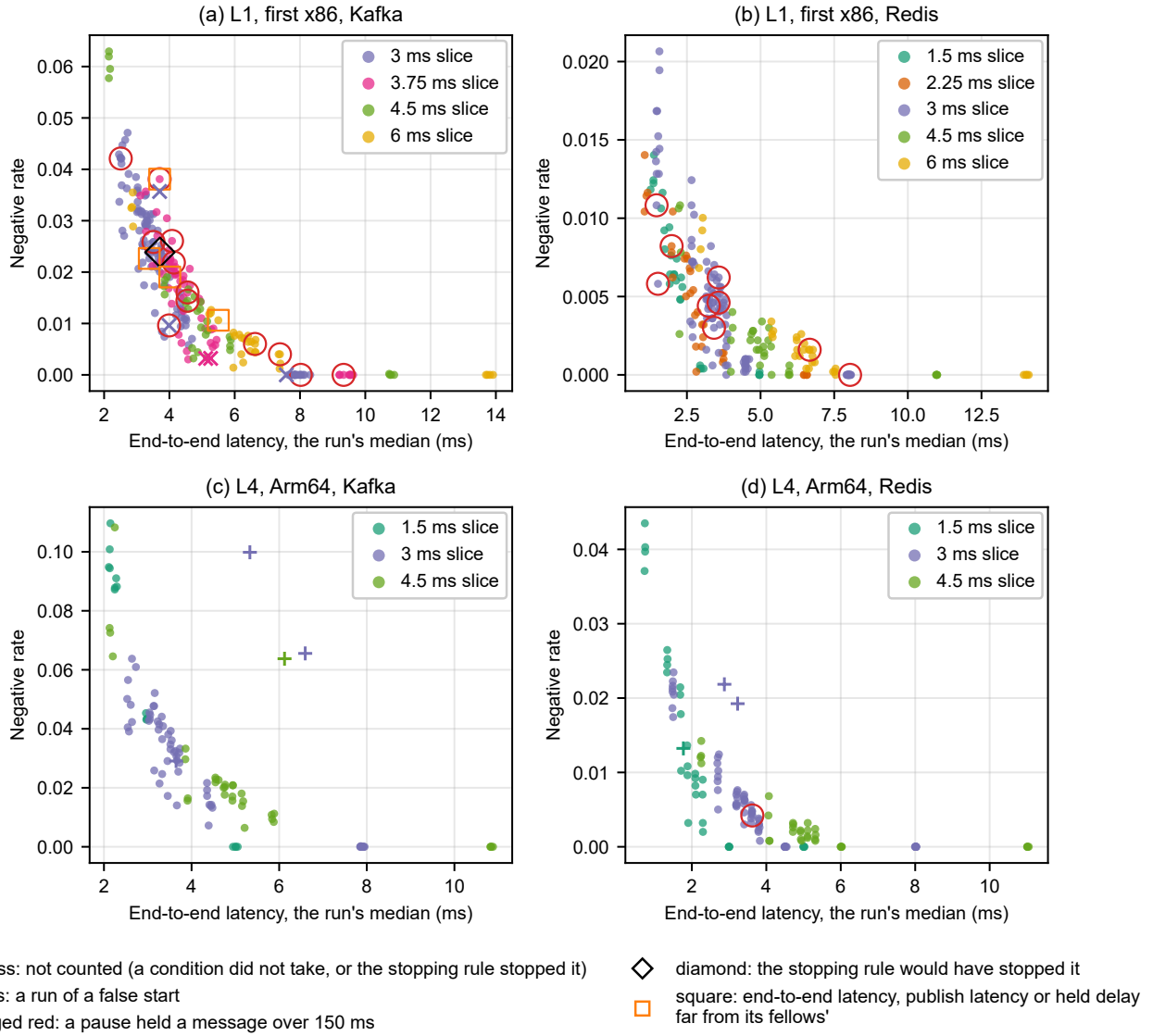


Fig. S4. **Slice runs fall from the shortest end-to-end latency on, where the registered law put a plateau.** Every run of L1 (first x86 pair) and L4 (Arm pair, Arm64 in the panels) at its median end-to-end latency and its negative-span rate, colored by the slice it ran at. Crosses: runs the integrity rule did not count. Plusses: runs of L4's two false starts. Rings, diamonds and squares: counted runs that are paused, flagged, or far from their setup's other repeats. That the rate holds no plateau was read after the verdicts (S6.3).

either because the receiving side's disk stalled, for a cause not identified, or because Kafka's broker could not reach its own controller. No run had a message arrive before it was published. Without the flagged runs, or without all 1,110 taken while the stopping rule only recorded, no verdict moves. Without the paused runs every verdict stands but P3a on the first x86 pair, whose fitted interval sits on the edge of its ± 0.25 ms bar, just outside it with them and just inside without. The verdict on all the runs is the one reported. The verdicts, on all runs and on each subset, are under `docs/results/law/judged-27-sep`.

S6.3. Read after the verdicts: the wait is the rest of a slice, and two judging programs departed from the plan

Everything here was read after the verdicts, and none of it changes one. The rate holds no plateau. At half the slice it is 1.45 to 3.94 times its value at 0.9 of the slice, in each of the ten slice and broker combinations whose runs reach half the slice.

L9's recordings, at a 3 ms slice and a 1 ms tick, time the wait of the thread that reads the clock for each acknowledgment, for 1,516,342 acknowledgments in 304 runs. Of those whose thread waited more than 1 ms, 24.3 to 35.8% were still waiting at 3 ms and 3.3 to 6.8% past 4 ms, over twelve pair, broker and load parts. A thread preempted for a task starting its slice waits out all of it, but a thread woken while another task holds the core waits out what is left. The acknowledgment's reply wakes the thread that reads it, so its wait is the second kind. A whole-slice wait would leave all of them still waiting at 3 ms; a

wait for the rest of the slice leaves 20% waiting at 3 ms and none past 4 ms. That thread waits out what is left of the running task's slice, R , uniform on $[0, s]$, and then up to a tick, U , uniform on $[0, h]$, so that for $h \leq s$

$$G(x) = \Pr[R + U > x] = \begin{cases} 1 - x^2/(2sh), & 0 \leq x \leq h, \\ 1 - (x - h/2)/s, & h \leq x \leq s, \\ (s + h - x)^2/(2sh), & s \leq x \leq s + h, \\ 0, & x \geq s + h. \end{cases} \quad (\text{S3})$$

The recording raised the rate it recorded, 1.36 to 1.49 times for Kafka and 3.69 to 4.33 times for Redis, so these waits describe the machines with it on. Fitted to a rate that falls from the shortest end-to-end latency on, the law's shape puts its fall below the slice and moves it less than the slice moves, which is why the free and fitted readings of P1 and P9 split.

In two places a frozen program departed from the plan, and in both the frozen answer is the one reported. The programs judging P2, P2b and P2c group L2's runs by tick alone, so each width they read spans the falls of both its slices, 1.5 and 3 ms. With Equation S2 in place of every run's rate, as if the law held exactly, the P2 program returns 1.98 free and 1.57 fitted for Redis, below its band. One slice at a time, the same formula gives 4.0 to 5.2 for P2 and 9.8 to 12.0 for P2b, inside both bands. Those three verdicts therefore carry no evidence about the law either way. P4's program words Redis "not confirmed" although its plateau, 0.69%, lies below the 2% at which P4 makes a claim, and the plan's rule reports a plateau that low as out of reach.

REFERENCES

- [S1] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023. [Online]. Available: <https://github.com/statsbomb/open-data>
- [S2] Linux Kernel Documentation, "KVM-specific MSRs: MSR_KVM_SYSTEM_TIME_NEW," <https://docs.kernel.org/virt/kvm/x86/msr.html>, 2026, accessed: Aug. 21, 2026.
- [S3] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit 2ae891b82695, 2025.
- [S4] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766.
- [S5] Apache Software Foundation, "Apache Kafka tools — EndToEndLatency.java and ProducerPerformance.java," Source repository, 2026, <https://github.com/apache/kafka>. Accessed: Aug. 31, 2026.
- [S6] Apache Pulsar contributors, "PerformanceConsumer.java," Source repository, 2026, <https://github.com/apache/pulsar>. Accessed: Aug. 21, 2026.
- [S7] Synadia, "NATS cli — cli/latency_command.go," Source repository, 2026, <https://github.com/nats-io/natscli>. Accessed: Oct. 2, 2026.
- [S8] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.
- [S9] Broadcom, "RabbitMQ PerfTest — TimestampProvider.java," Source repository, 2026, <https://github.com/rabbitmq/rabbitmq-perf-test>. Accessed: Oct. 2, 2026.
- [S10] IOP Systems, "Rezulus: systems performance telemetry — scheduler_discarded_samples," Source repository, 2026, <https://github.com/iopsystems/rezulus>. Accessed: Aug. 20, 2026.
- [S11] A. D. Brunelle, "blktrace/btt — btt/inlines.h," Source repository, 2026, accessed: Aug. 20, 2026.
- [S12] EMQ Technologies, "emqtt-bench: MQTT benchmark tool — src/emqtt_bench.erl," Source repository, 2026, <https://github.com/emqx/emqtt-bench>. Accessed: Aug. 21, 2026.
- [S13] J. Axboe, "fio: Flexible I/O tester — gettime.c," Source repository, 2026, <https://github.com/axboe/fio>. Accessed: Aug. 20, 2026.
- [S14] G. Tene, "wrk2 — src/wrk.c," Source repository, 2026, <https://github.com/giltene/wrk2>. Accessed: Aug. 21, 2026.